Getting Started with RSnowflake

RSnowflake is a DBI-compliant R interface to Snowflake that connects directly to the Snowflake SQL API via HTTPS. It has no dependency on ODBC, JDBC, or Python.

Installation

```
# From GitHub (development version):
# install.packages("pak")
pak::pak("Snowflake-Labs/RSnowflake")
```

Authentication

RSnowflake supports several authentication methods. The recommended approach is to configure a profile in `~/.snowflake/connections.toml`, the same file used by the Snowflake Python connector and Snowflake CLI.

connections.toml (recommended)

```
[default]
account   = "myaccount"
user      = "myuser"
authenticator = "SNOWFLAKE_JWT"
private_key_path = "/path/to/rsa_key.p8"
database  = "MY_DB"
schema    = "PUBLIC"
warehouse = "MY_WH"
```

Then connect without any arguments:

```
library(DBI)
library(RSnowflake)

con <- dbConnect(Snowflake())
```

Or select a named profile:

```
con <- dbConnect(Snowflake(), name = "my_profile")
```

Programmatic Access Token (PAT)

A **Programmatic Access Token** is useful for automation, CI, or hosts where you do not want to store a long-lived private key. Create a PAT in **Snowsight > Admin > Security > Programmatic Access Tokens** (wording may vary by account), then either set `SNOWFLAKE_PAT` for the R process or pass `token =` explicitly. Treat tokens like passwords: prefer environment variables or a secrets manager, and never commit them to git.

```
# Set SNOWFLAKE_PAT in the shell or secrets manager before starting R.

con <- dbConnect(
```

```
  Snowflake(),
  account = "myaccount",
  token   = Sys.getenv("SNOWFLAKE_PAT")
)
```

PATs use Snowflake's `PROGRAMMATIC_ACCESS_TOKEN` authentication path in the driver. In **Snowflake Workspace Notebooks**, the managed **SPCS OAuth** session is usually preferred for `dbConnect(Snowflake())`; reserve PATs for REST-style or external tooling that cannot use the Workspace session.

Snowflake Workspace Notebooks

In Workspace you run R inside **`%%R` cells** on a **Python kernel** (via rpy2). RSnowflake still speaks HTTPS to Snowflake; the Workspace runtime injects the same **session / OAuth** material the Snowpark Python client uses.

For zero-config DBI from R, run a small Python bootstrap cell first. The **`setup_notebook()`** function from **`sfnb_setup.py`** (bundled with the `snowflake-notebook-multilang` toolkit and with **`snowflakeR`**'s `inst/notebooks/` examples) installs R, sets session context, and **exports environment variables** so `Snowflake()` can connect without a `connections.toml` file, for example:

- `SNOWFLAKE_ACCOUNT`, `SNOWFLAKE_USER`, `SNOWFLAKE_HOST`
- `SNOWFLAKE_ROLE`, `SNOWFLAKE_WAREHOUSE`, `SNOWFLAKE_DATABASE`, `SNOWFLAKE_SCHEMA` when your bootstrap or session defines them

After the Snowpark session exists in Python, open R and connect:

```
library(DBI)
library(RSnowflake)

con <- dbConnect(Snowflake())
```

If `dbConnect()` fails from a `%%R` cell, confirm the **Python** setup cell ran successfully and that you are not mixing profiles intended for your laptop with the Workspace session. For ML + notebook ergonomics (context, EAI, `%%R` patterns), see **`vignette("workspace-notebooks", package = "snowflakeR")`**.

For **EAI**, **Go / adbcsnowflake installs**, and the internal SPCS gateway in more detail, see **`vignette("workspace-rsnowflake", package = "RSnowflake")`**.

Running Queries

```
# Simple query
df <- dbGetQuery(con, "SELECT * FROM my_table LIMIT 10")

# Parameterized query
df <- dbGetQuery(
  con,
  "SELECT * FROM users WHERE age > ?",
  params = list(21L)
)

# Execute DDL / DML (returns rows affected)
dbExecute(con, "CREATE TABLE test_tbl (id INT, name VARCHAR)")
dbExecute(con, "INSERT INTO test_tbl VALUES (1, 'Alice')")
```

Writing and Reading Tables

```r
# Write a data.frame to Snowflake
dbWriteTable(con, "iris_copy", iris)

# Read it back
iris_back <- dbReadTable(con, "iris_copy")

# Append more rows
dbAppendTable(con, "iris_copy", iris[1:10, ])

# Clean up
dbRemoveTable(con, "iris_copy")
```

Bulk Write Methods

RSnowflake automatically picks the fastest write path for your environment. The `upload_method` option controls this:

Method	How it works	Best for
`"auto"` (default)	Snowpark in Workspace, ADBC outside, literal for small data	Most users
`"snowpark"`	Python `write_pandas` via reticulate (internal SPCS path)	Workspace bulk writes
`"adbc"`	Arrow-native PUT + COPY INTO via Go driver	Local/external bulk writes
`"literal"`	SQL INSERT statements	Small data, maximum compatibility

In **Workspace Notebooks**, `"auto"` routes bulk writes (above `RSnowflake.bulk_write_threshold` cells) through the Python Snowpark session via reticulate. This uses the internal SPCS network path and is ~10x faster than ADBC for writes in Workspace (~2.5s vs ~21s for 50K rows).

Outside Workspace, `"auto"` routes to ADBC when available, which outperforms literal INSERT for datasets above ~20K rows.

```r
# Force a specific method
options(RSnowflake.upload_method = "snowpark")
dbWriteTable(con, "big_table", large_df)

# Adjust the threshold (cells = rows * cols)
options(RSnowflake.bulk_write_threshold = 100000L)
```

Identifier Case Handling

By default, RSnowflake uppercases table and column names in DDL/DML operations (`dbWriteTable`, `dbCreateTable`, `dbReadTable`, etc.). This matches the behavior of Snowflake's ODBC driver and of Snowflake itself when identifiers are unquoted:

```r
dbWriteTable(con, "my_table", data.frame(id = 1, name = "Alice"))
dbListFields(con, "my_table")
#> [1] "ID" "NAME"
```

To preserve the original case of identifiers (useful for DBItest compliance or when you need lowercase column names), set:

```r
options(RSnowflake.identifier_case = "preserve")

dbWriteTable(con, "my_table", data.frame(id = 1, name = "Alice"), overwrite = TRUE)
dbListFields(con, "my_table")
#> [1] "id" "name"
```

Note: `dbQuoteIdentifier()` is not affected by this option – it always faithfully quotes the string you give it. The option only controls the implicit casing applied in table-level operations.

ADBC Acceleration (optional)

When the **`adbcsnowflake`** and **`adbcdrivermanager`** packages are installed (and on Linux/macOS the **Go toolchain** is available for building the Go driver), RSnowflake can use **ADBC** (Arrow Database Connectivity) for Arrow-native reads and **PUT + COPY INTO** bulk writes.

Where it helps

Environment	Typical behaviour
Local / Posit / CI	`upload_method = "auto"` often picks ADBC for large `dbWriteTable()` / `dbAppendTable()` when the driver is installed; good for wide tables and Arrow-heavy pipelines.
Workspace Notebooks	ADBC (when installed) can use the **internal SPCS gateway** (`SNOWFLAKE_HOST`) with the container OAuth token – no public `*.snowflakecomputing.com` URL is required for Snowflake traffic. Bulk writes may still route via Snowpark (`write_pandas`) when `"auto"` prefers that path; see the table in **Bulk Write Methods** above.

```r
# Check if ADBC is available
has_adbc <- requireNamespace("adbcsnowflake", quietly = TRUE)
cat("ADBC available:", has_adbc, "\n")

# ADBC participates automatically in 'auto' for large writes when installed
# Force the query backend for reads:
options(RSnowflake.backend = "adbc")
df <- dbGetQuery(con, "SELECT * FROM big_table")
options(RSnowflake.backend = "auto")
```

Workspace constraints (EAI / network rules for installing `adbcsnowflake`, Go module download hosts, internal gateway behaviour) are documented in **`WORKSPACE_ADBC.md`** at the root of the RSnowflake package source (same content in the GitHub repository). Read that file before enabling ADBC inside locked-down Workspace images.

Arrow Interface

RSnowflake supports the DBI Arrow methods for compatibility with packages and workflows that expect them. This requires the `nanoarrow` package.

```r
stream <- dbGetQueryArrow(con, "SELECT * FROM my_table")
df <- as.data.frame(stream)
```

Note: The Snowflake SQL API v2 only returns JSON, so these Arrow methods fetch data through the normal JSON path and convert to nanoarrow on the client side. This provides interface compatibility but no performance advantage over `dbGetQuery()`. For large result sets, `dbGetQuery()` is the most direct path. Native server-side Arrow transport is planned for a future release.

dbplyr Integration

RSnowflake integrates with dbplyr so you can use dplyr verbs that are translated to Snowflake SQL and executed lazily:

```r
library(dplyr)

tbl(con, "my_table") |>
  filter(score > 90) |>
  select(name, score) |>
  arrange(desc(score)) |>
  collect()
```

Snowflake-Specific SQL Translations

In addition to the ~150 scalar and ~30 aggregate translations that dbplyr provides for Snowflake (e.g. `floor_date()` to `DATE_TRUNC`, `paste0()` to `ARRAY_TO_STRING`, `ifelse()` to `IFF`), RSnowflake adds translations for 17 Snowflake functions that dbplyr does not cover. These let you use Snowflake-specific SQL functions directly in dplyr pipelines without falling back to `sql()`.

Semi-structured data – work with VARIANT, OBJECT, and ARRAY columns:

R function	Snowflake SQL	Description
`parse_json(x)`	`PARSE_JSON(x)`	Parse a JSON string into VARIANT
`try_parse_json(x)`	`TRY_PARSE_JSON(x)`	Safe parse (returns NULL on error)
`typeof(x)`	`TYPEOF(x)`	Inspect the type of a VARIANT value
`is_object(x)`	`IS_OBJECT(x)`	Check if VARIANT is an OBJECT
`is_array(x)`	`IS_ARRAY(x)`	Check if VARIANT is an ARRAY
`is_integer(x)`	`IS_INTEGER(x)`	Check if VARIANT is an INTEGER

Array operations:

R function	Snowflake SQL	Description
`array_size(x)`	`ARRAY_SIZE(x)`	Length of an ARRAY
`array_contains(arr, val)`	`ARRAY_CONTAINS(val, arr)`	Test membership (note: arg order swapped for you)
`array_slice(arr, from, to)`	`ARRAY_SLICE(arr, from, to)`	Slice an ARRAY

String and utility:

R function	Snowflake SQL	Description
`regexp_substr(x, pattern)`	`REGEXP_SUBSTR(x, pattern, ...)`	Extract substring by regex (supports all 5 Snowflake args)
`hash(...)`	`HASH(...)`	Deterministic hash of one or more columns

Aggregate functions – use inside `summarise()`:

R function	Snowflake SQL	Description
`object_agg(key, value)`	`OBJECT_AGG(key, value)`	Build a JSON object from grouped key-value pairs
`array_agg(x)`	`ARRAY_AGG(x)`	Collect values into an ARRAY
`array_unique_agg(x)`	`ARRAY_UNIQUE_AGG(x)`	Collect distinct values into an ARRAY
`approx_count_distinct(x)`	`APPROX_COUNT_DISTINCT(x)`	HyperLogLog approximate distinct count
`approx_percentile(x, p)`	`APPROX_PERCENTILE(x, p)`	Approximate percentile
`mode_val(x)`	`MODE(x)`	Statistical mode (`mode_val` avoids shadowing `base::mode()`)

```r
# Example: parse JSON and aggregate into arrays
tbl(con, "events") |>
  mutate(payload = parse_json(raw_json)) |>
  group_by(event_type) |>
  summarise(
    user_ids = array_unique_agg(user_id),
    n_approx = approx_count_distinct(user_id)
  ) |>
  collect()
```

For the full list of dbplyr's built-in translations, see `vignette("translation-function", package = "dbplyr")`.

Transactions

Note: The Snowflake SQL API v2 is stateless per-request and does not currently support session-based transactions. Calling `dbBegin()` will raise an informative error. Session-based transaction support is planned for a future release that uses the internal Snowflake protocol.

```r
# Not yet supported via SQL API v2 -- included for reference:
dbBegin(con)
dbExecute(con, "INSERT INTO accounts VALUES (1, 100.00)")
dbCommit(con)
```

Disconnecting

```r
dbDisconnect(con)
```